

EX NO:01	
DATE:	

AIM:

To create a student dataset, load it using the Pandas library, display the shape of the dataset, view the data types of each column, display the first 10 records, and identify missing values along with their percentage.

ALGORITHM:

- 1.Import the required libraries.
- 2.Create a CSV file containing student data.
- 3.Load the dataset using Pandas.
- 4.Display the first 10 records of the dataset.
- 5.Display the shape of the dataset.
- 6.Display the data type of each column.
- 7.Plot the semester marks.
- 8.Calculate the missing values in each column.
- 9.Calculate the percentage of missing values.
- 10.Display the results.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv(r"C:\AML\student_data.csv")
print("First 10 Records")
print(df.head(10))
print("\nShape of Dataset:")
print(df.shape)
print("\nData Types:")
print(df.dtypes)
plt.figure(figsize=(10,5))
plt.bar(df["Student_ID"], df["Semester_Mark"])
plt.title("Semester Marks of Students")
plt.xlabel("Student ID")
plt.ylabel("Semester Marks")
plt.show()
df = pd.read_csv(r"C:\AML\student_dataset.csv")
```

OUTPUT:

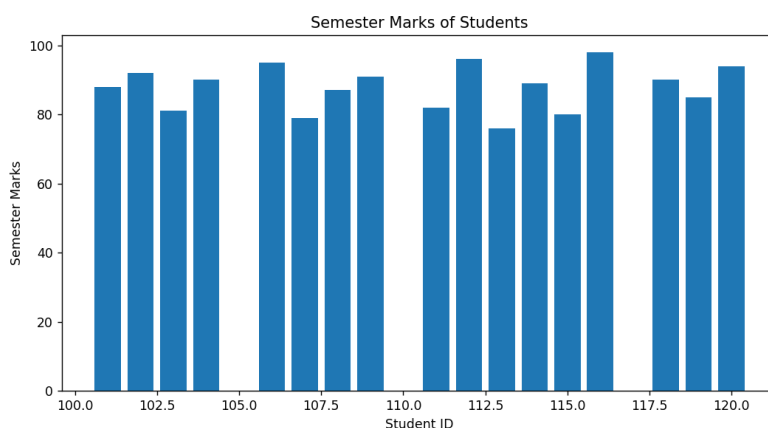
First 10 Records

	Student_ID	Student_Name	Gender	Age	Attendance	Internal_Mark	Semester_Mark
0	101	Aarav	Male	18.0	91.0	45.0	88.0
1	102	Diya	Female	19.0	95.0	48.0	92.0
2	103	Rohan	Male	NaN	87.0	42.0	81.0
3	104	Priya	Female	19.0	93.0	NaN	90.0
4	105	Arjun	Male	18.0	89.0	44.0	NaN
5	106	Sneha	Female	20.0	96.0	49.0	95.0
6	107	Karthik	Male	19.0	NaN	40.0	79.0
7	108	Meera	Female	18.0	90.0	45.0	87.0
8	109	Vikram	Male	20.0	92.0	47.0	91.0
9	110	Ananya	Female	18.0	94.0	48.0	NaN

Shape of Dataset:
(20, 7)

Data Types:

Student_ID int64
Student_Name str
Gender str
Age float64
Attendance float64
Internal_Mark float64
Semester_Mark float64
dtype: object



```
print("\nMissing Values in Each Column:")  
print(df.isnull().sum())
```

```

missing_percentage = (df.isnull().sum() / len(df)) * 100
print("\nPercentage of Missing Values:")
print(missing_percentage)plt.show()
df = pd.read_csv(r"C:\AML\sstudent_dataset.csv")
print("\nMissing Values in Each Column:")
print(df.isnull().sum())
missing_percentage = (df.isnull().sum() / len(df)) * 100
print("\nPercentage of Missing Values:")
print(missing_percentage)

```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The Student Performance dataset was successfully loaded, and Min-Max Normalization was applied to the Study_Hours, Attendance, and Assignment_Marks features. The values were scaled to the range 0 to 1, and the comparison along with histograms confirmed the successful normalization of the selected features.

Missing values in each column:

Student - ID 0

EX NO:02

DATE:

Student - Name 0

Gender 0

Age 2

Attendance 1

Internal - Marks 2

Session - Mark 3

dtype: int64

AIM:

To estimate the mean marks of students using Maximum A Posteriori (MAP) estimation by assuming a Gaussian prior distribution.

ALGORITHM:

- 1.Import the required libraries.
- 2.Load the student marks dataset.
- 3.Calculate the Maximum Likelihood Estimation (MLE) mean.
- 4.Assume a Gaussian prior.
- 5.Compute the MAP estimate.
- 6.Compare the MAP estimate with the MLE mean.
- 7.Display the results.

CODE:

```
import pandas as pd
import numpy as np
df = pd.read_csv(r"C:\AML\ex2.csv")
marks = df["Marks"]
mle_mean = np.mean(marks)
prior_mean = 70
prior_variance = 25
sample_variance = np.var(marks)
n = len(marks)
map_mean = ((prior_mean/prior_variance) + (n*mle_mean/sample_variance)) /
((1/prior_variance) +(n/sample_variance))
print("MLE Mean :", mle_mean)
print("MAP Mean :", map_mean)
```

OUTPUT:

MLE Mean : 72.8

MAP Mean : 72.10650488709133

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the student marks dataset was successfully analyzed using MAP estimation. The MAP estimate was computed using prior information and compared with the MLE mean successfully.

EX NO:03	
DATE:	

AIM:

To apply Min–Max normalization on a student performance dataset and scale the selected features to the range of 0 to 1.

ALGORITHM:

- 1.Import the required libraries.
- 2.Load the student dataset.
- 3.Select the required numerical features.
- 4.Find the minimum and maximum values of each feature.

5. Apply the Min–Max normalization formula.
6. Normalize all selected features.
7. Display the normalized dataset.
8. Plot the normalized values.
9. Compare the original and normalized data.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
df = pd.read_csv(r"C:\AML\ex3.csv")
print("Original Data")
print(df.head())
features = ["Study_Hours", "Attendance", "Assignment_Marks"]
scaler = MinMaxScaler()
```

OUTPUT:

Original Data

	Student_ID	Study_Hours	Attendance	Assignment_Marks	Final_Marks
0	S001	2	70	55	58
1	S002	3	75	60	63
2	S003	4	78	62	65
3	S004	5	80	65	68
4	S005	6	82	68	70

Normalized Data

	Study_Hours	Attendance	Assignment_Marks
0	0.000000	0.000000	0.000000
1	0.090909	0.178571	0.135135
2	0.181818	0.285714	0.189189
3	0.272727	0.357143	0.270270
4	0.363636	0.428571	0.351351

	Original	...	Normalized		
	Study_Hours	Attendance	... Attendance	Assignment_Marks	
0	2	70	...	0.000000	0.000000
1	3	75	...	0.178571	0.135135
2	4	78	...	0.285714	0.189189
3	5	80	...	0.357143	0.270270
4	6	82	...	0.428571	0.351351

[5 rows x 6 columns]

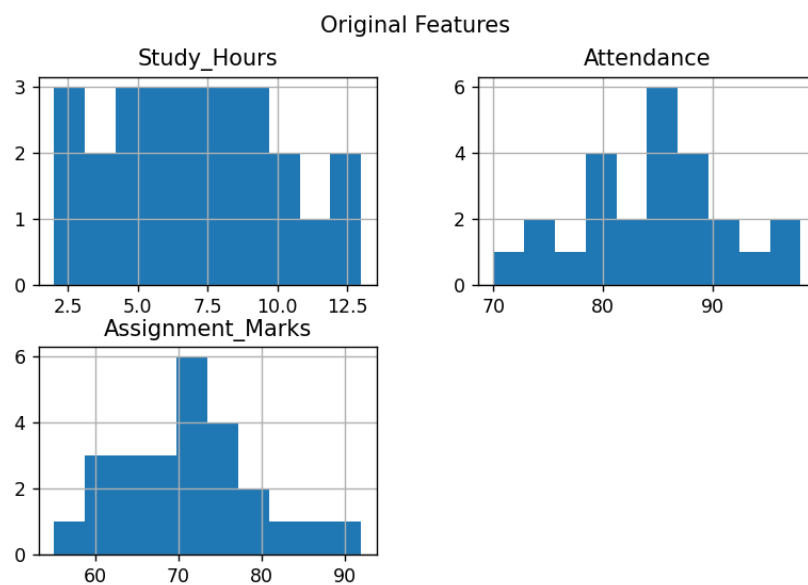
```
scaled = scaler.fit_transform(df[features])
scaled_df = pd.DataFrame(scaled, columns=features)
print("\nNormalized Data")
print(scaled_df.head())
comparison = pd.concat([df[features], scaled_df], axis=1, keys=["Original",
"Normalized"])
print(comparison.head())
df[features].hist(figsize=(8,4))
plt.suptitle("Original Features")
scaled_df.hist(figsize=(8,4))
plt.suptitle("Normalized Features")
```

```
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

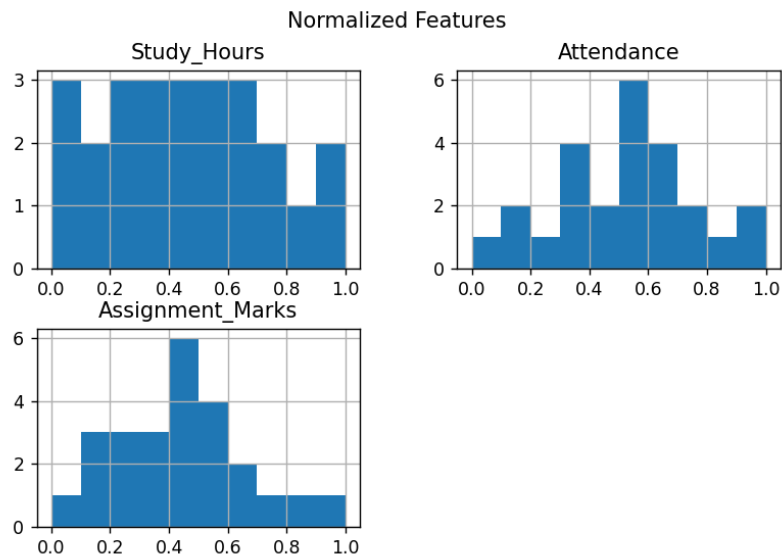
RESULT:

Thus, Min–Max normalization was successfully applied to the dataset, and all selected features were scaled to the range of 0 to 1 successfully.



EX NO:04

DATE:



AIM:

To estimate the mean and variance of a Gaussian distribution using Maximum Likelihood Estimation (MLE).

ALGORITHM:

- 1.Import the required libraries.
- 2.Load the student marks dataset.
- 3.Compute the sample mean.
- 4.Compute the sample variance.
- 5.Estimate the Gaussian distribution parameters using MLE.
- 6.Display the estimated mean and variance.
- 7.Compare the estimated values with the sample statistics..

CODE:

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
df = pd.read_csv(r"C:\AML\ex4.csv")
marks = df["Marks"]
mu = np.mean(marks)
sigma = np.sqrt(np.mean((marks - mu)**2))
print("MLE Mean:", mu)
print("MLE Standard Deviation:", sigma)
mu2, sigma2 = norm.fit(marks)
print("Scipy Mean:", mu2)
print("Scipy Standard Deviation:", sigma2)
plt.hist(marks, bins=10, density=True, alpha=0.6)
```

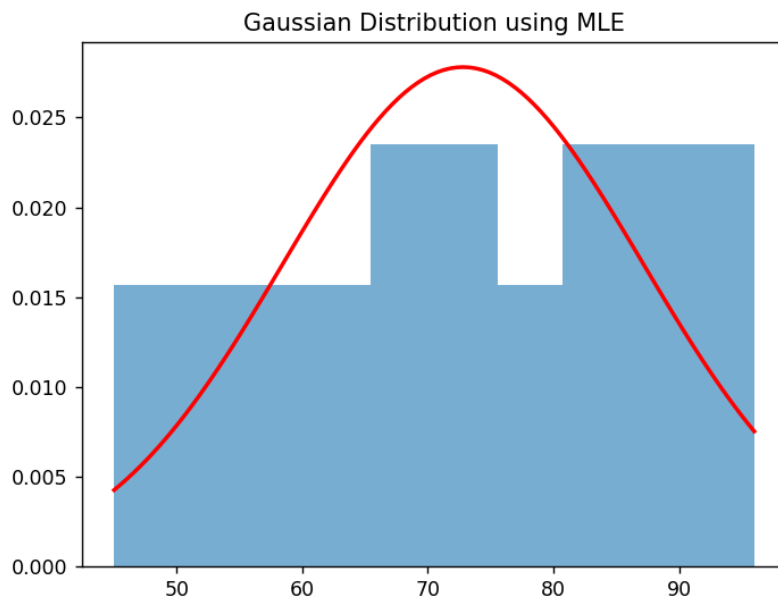
OUTPUT:

MLE Mean: 72.8

MLE Standard Deviation: 14.344336861632886

Scipy Mean: 72.8

Scipy Standard Deviation: 14.344336861632886



```
x = np.linspace(min(marks), max(marks), 100)
plt.plot(x, norm.pdf(x, mu, sigma), 'r', linewidth=2)
plt.title("Gaussian Distribution using MLE")
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the Gaussian distribution parameters (mean and variance) were successfully estimated using the Maximum Likelihood Estimation (MLE) method.

EX NO:05	
DATE:	

AIM:

To predict the price of used cars using Linear Regression implemented with the Gradient Descent optimization algorithm.

ALGORITHM:

- 1.Import the required libraries.
- 2.Load the used car dataset.
- 3.Convert categorical features into numerical values.
- 4.Select the input features and target variable.

5. Normalize the input features.
6. Initialize the Linear Regression model.
7. Train the model using Gradient Descent.
8. Predict the car prices.
9. Compare the actual and predicted prices.
10. Plot the Mean Squared Error (MSE) reduction graph.
11. Display the results.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
data = pd.read_csv(r"C:\AML\ex5.csv")
data["Brand"] = data["Brand"].astype("category").cat.codes
data["Fuel_Type"] = data["Fuel_Type"].astype("category").cat.codes
X = data[["Year", "Mileage", "Engine_Size"]].values
y = data["Price"].values
X = (X - X.mean(axis=0)) / X.std(axis=0)
class LinearRegressionGD:
    def __init__(self, learning_rate=0.01, epochs=1000):
```

OUTPUT:

	Actual Price	Predicted Price
0	850000	797141
1	900000	779178
2	700000	662348
3	750000	877558
4	600000	600382
5	1200000	1212174
6	850000	804244

7	950000	940458
8	500000	498938
9	1100000	1198696
10	550000	505554
11	1150000	1094411
12	1300000	1228493
13	700000	782730
14	780000	799272
15	1600000	1481760
16	580000	603933
17	760000	760728
18	900000	973096
19	980000	1098896

```
self.lr = learning_rate
self.epochs = epochs
def fit(self, X, y):
    m, n = X.shape
    self.weights = np.zeros(n)
    self.bias = 0
    self.loss_history = []
    for i in range(self.epochs):
        y_pred = np.dot(X, self.weights) + self.bias
        error = y_pred - y
        mse = np.mean(error ** 2)
```

```

self.loss_history.append(mse)

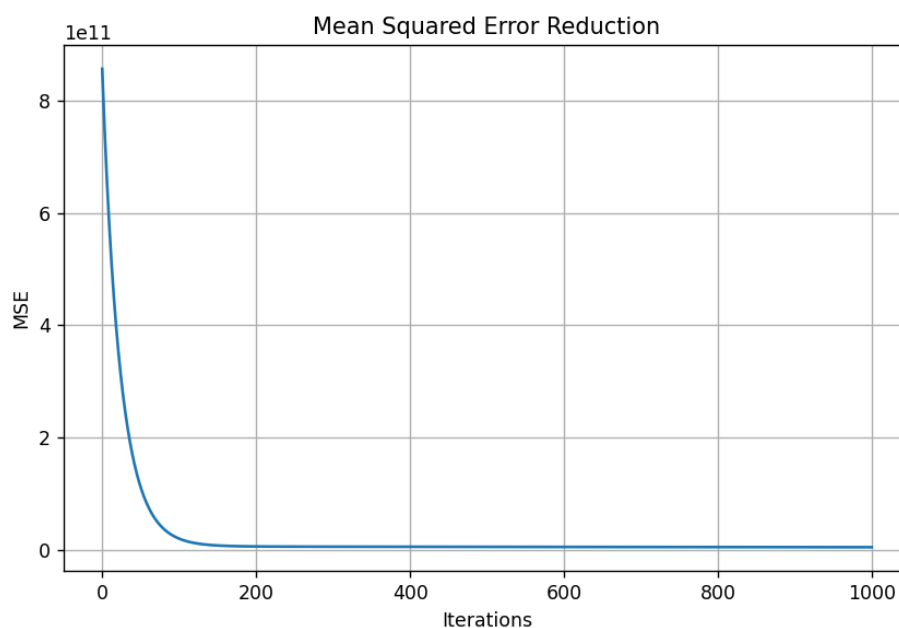
dw = (2/m) * np.dot(X.T, error)
db = (2/m) * np.sum(error)

self.weights -= self.lr * dw
self.bias -= self.lr * db

def predict(self, X):
    return np.dot(X, self.weights) + self.bias

model = LinearRegressionGD(learning_rate=0.01, epochs=1000)
model.fit(X, y)
predictions = model.predict(X)
result = pd.DataFrame({
    "Actual Price": y.astype(int),
    "Predicted Price": predictions.astype(int)
})
print(result)
plt.figure(figsize=(8,5))
plt.plot(model.loss_history)

```



```
plt.title("Mean Squared Error Reduction")
plt.xlabel("Iterations")
plt.ylabel("MSE")
plt.grid(True)
plt.show()

x = np.linspace(min(marks), max(marks), 100)
plt.plot(x, norm.pdf(x, mu, sigma), 'r', linewidth=2)
plt.title("Gaussian Distribution using MLE")
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the Gaussian distribution parameters (mean and variance) were successfully estimated using the Maximum Likelihood Estimation (MLE) method.

